Implement Digital Signatures

RSA Signatures · Certificate Management · Signature Verification · Trust Model

A clinical document signing system for HVMC: RSA-PSS and PKCS #1 v1.5 written from scratch and verified against the cryptography library and OpenSSL, a time-stamping authority, certificate revocation and renewal, and a 13-scenario verification test — 14 of 14 tests passed

Project Name	Healthcare IAM Operations Simulation
Organization	Horizon Valley Medical Center (HVMC)
Rubric	Cryptography 1 — Implement Digital Signatures
Code	`cryptography_1/digital_signatures/` : `signatures.py`, `cert_management.py`, `signing_system.py`, `test_and_demo.py`, `sample_documents/`, `signed_documents/`, `pki/`, `trust_store.json`, `results.txt`, `openssl_checks.txt`
Builds on	The two-tier HVMC PKI and from-scratch RSA from *Utilize Public Key Cryptography* (`cryptography_1/public_key_crypto/`)
Environment	Python 3.9.6 on Darwin arm64, 2026-10-26 13:57; cryptography package: 47.0.0; LibreSSL 3.3.6
Prepared By	Sabitha S Karanavar
Report Date	October 26, 2026

Rubric Cross-Reference

Rubric requirement	Section	Evidence
Creation of a digital signature system	2	HVMC clinical document signing system: signers, detached signature envelopes, a time-stamping authority, CRL publication and a relying-party verifier; 7 clinical documents signed
Implementation of RSA-based digital signatures	3	RSASSA-PSS and RSASSA-PKCS1-v1_5 from scratch (RFC 8017); PSS accepted by the library and OpenSSL; PKCS #1 v1.5 byte-identical to both; textbook-RSA forgery shown
Certificate generation and management	4	Document-signing and time-stamping profiles; 5 certificates issued with approver; contractor end-date cap; revocation with reason codes and invalidity date; re-key; renewal monitor; CRLs checked by OpenSSL
Signature verification process	5	12-step verifier with VALID / INVALID / INDETERMINATE results; 13 scenarios including tampering, impostor CA, revocation timing, key compromise and long-term validation
Documentation of the trust model implementation	6	Hierarchical PKI with pinned anchor, roles, certificate policy OID, authorisation matrix, time and revocation rules, comparison with other trust models, threats and mitigations

1. Introduction

A **digital signature** is the electronic equivalent of a handwritten signature and a tamper seal combined. The signer uses a **private key** to sign; anyone with the matching **public key** — delivered in a certificate — can check it. A valid signature proves three things:

Property	Meaning at HVMC
Integrity	The discharge summary or medication verification has not changed by a single byte since it was signed
Authenticity	It was signed by the holder of Sarah Chen's signing key, whose identity HVMC's CA checked
Non-repudiation	Sarah cannot later deny signing it: only she holds the key, and a trusted timestamp fixes when she signed

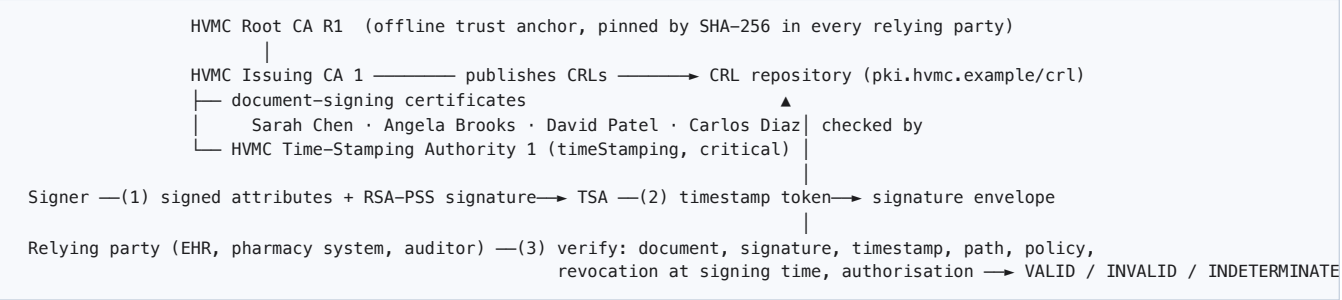
HIPAA requires mechanisms to protect ePHI from improper alteration and to **corroborate that ePHI has not been altered** (§164.312(c)(1)–(2)), and **person or entity authentication** (§164.312(d)). Pharmacy and research records also fall under FDA **21 CFR Part 11**, which requires electronic signatures to be linked to their records so they cannot be copied or transferred to falsify a record (§11.70) — exactly what binding the document hash and signer certificate into the signed attributes achieves.

This project builds on the HVMC PKI from the *Utilize Public Key Cryptography* report. The test script copies that PKI, so the earlier report's evidence is unchanged, and adds signing certificates, a time-stamping authority and a full signing and verification system.

14/14	5	3	13	3
unit tests passed	signing & TSA certificates issued	certificates revoked with reasons	verification scenarios (5 valid)	implementations agree (mine, library, OpenSSL)

2. The HVMC Document Signing System

2.1 Components



Component	Code	Role
Signer	`signing_system.Signer`	Holds a clinician's document-signing key (RSA-3072); builds the signed attributes and signs them with RSA-PSS
Time-stamping authority	`signing_system.TimestampAuthority`	Countersigns a hash of each signature with its own clock time (simplified RFC 3161), proving *when* it existed
Certificate authority	`cert_management.ManagedCA`	The rubric 4 issuing CA, extended to issue signing/TSA certificates, revoke, re-key and publish CRLs
Trust store	`signing_system.TrustStore`, `trust_store.json`	What a relying party trusts: the pinned root, the required certificate policy, accepted algorithms, who may sign what
Verifier	`signing_system.Verifier`	The verification process in section 5

2.2 The signature envelope

HVMC uses **detached signatures**: the clinical document stays unchanged in the EHR and the signature is stored beside it (`signed_documents/*.sig.json`). Following CMS (RFC 5652) and CAdES, the signer does not sign the document directly but a set of **signed attributes** that contains the document's SHA-256 hash. This binds extra facts into the signature — the document ID and type, the exact signer certificate, the claimed time and the commitment type ("proof of approval") — so none of them can be changed or swapped later. Before signing, the attributes are put in a single **canonical form** (sorted keys, no whitespace), so signer and verifier hash exactly the same bytes.

The envelope for Sarah Chen's signed discharge summary (long values shortened):


```
{
  "format": "HVMC-detached-signature-v1",
  "algorithm": "RSASSA-PSS-SHA256",
  "signed_attributes": {
    "doc_id": "DS-HVMC-004417-01",
    "doc_type": "discharge-summary",
    "doc_sha256": "fd779d4597eb9cb994d4c70d7d3b6906f329e6b39dd091a92bd33bfa291e7186",
    "signer_cert_sha256": "be66e5e0da0495f369c03d482f1349d795618147809O6f5d1167f85a7abd615f",
    "claimed_signing_time": "2026-10-26T18:57:25+00:00",
    "commitment_type": "proof-of-approval",
    "content_type": "application/json"
  },
  "signature": "jL4XR49jJX2PqMIegoYKSbPYaKv4KCOSqU0B5CCsel92…(384 bytes, base64)",
  "certificates": [
    "MIIF5zCC…(Sarah Chen document-signing certificate, DER, base64)",
    "MIIF…(HVMC Issuing CA 1)"
  ],
  "timestamp": {
    "tst_info": {
      "version": 1,
      "policy": "2.25.301168339931612…",
      "hash_alg": "sha256",
      "message_imprint": "c75bdc17775a0b5dd558e58044e8886f3c17419ea667abb49ab2356c29031699",
      "gen_time": "2026-10-26T18:57:25+00:00",
      "serial": 1,
      "nonce": "3744b4c73d7412c6",
      "tsa": "HVMC Time-Stamping Authority 1"
```

```
    },
    "signature": "T/Y/HLtFgKI+wZqeKjaBAo0VLGy+Ig…",
    "tsa_certificate": "MIIF…(HVMC Time-Stamping Authority 1)"
  }
}
```

```
class Signer:
    def __init__(self, key, cert, chain, tsa=None):
        self.key, self.cert, self.chain, self.tsa = sig.scratch_private(key), cert, chain, tsa

    def sign(self, document, doc_id, doc_type, at, commitment="proof-of-approval", timestamp=True):
        attrs = {
            "doc_id": doc_id, "doc_type": doc_type, "doc_sha256": hashlib.sha256(document).hexdigest(),
            "signer_cert_sha256": self.cert.fingerprint(hashes.SHA256()).hex(),
            "claimed_signing_time": at.isoformat(), "commitment_type": commitment,
            "content_type": "application/json",
        }
        signature = sig.pss_sign(self.key, canonical(attrs))
        env = {"format": "HVMC-detached-signature-v1", "algorithm": ALG, "signed_attributes": attrs,
               "signature": b64(signature), "certificates": [b64(der(c)) for c in [self.cert] + self.chain]}
        if timestamp and self.tsa:
            env["timestamp"] = self.tsa.stamp(hashlib.sha256(signature).digest(), at)
        return env
```

3. RSA-Based Digital Signatures

3.1 How an RSA signature works

Signing is the RSA private-key operation applied to an encoded **hash** of the message: $s = \mathrm{EM}^d \bmod n$. Verifying is the public-key operation: compute $s^e \bmod n$ and check that it is a correct encoding of the message's hash. Only the private-key holder can produce s; anyone can check it. Hashing first lets a 3072-bit key sign a document of any size.

Why "textbook" RSA signatures fail. Signing the raw message ($s = m^d$) is multiplicative: $(m_1 m_2)^d = {m_1}^d \cdot {m_2}^d$. The demo multiplies two genuine signatures into a valid signature on a message the key holder never signed (**existential forgery**), and $s = 1$ is a valid signature on $m = 1$ for every key. Hashing and padding (an **encoding**) destroy this structure.

3.2 RSASSA-PKCS1-v1_5

```
EM = 00 ‖ 01 ‖ FF FF … FF ‖ 00 ‖ DigestInfo( SHA-256 OID, SHA-256(M) )        s = EM^d mod n
```

The older scheme, still used in most X.509 certificates (including every certificate in the HVMC PKI). It is **deterministic**: the same key and message always give the same signature, which makes it a precise test — the from-scratch signature is **byte-for-byte identical** to the library's and to OpenSSL's. Verification rebuilds the whole expected encoding and compares it, rather than parsing it; lenient parsers were the cause of the 2006 Bleichenbacher signature-forgery bugs.

3.3 RSASSA-PSS (the HVMC default)

```
M'  = 00×8 ‖ SHA-256(M) ‖ salt                    salt = 32 random bytes, new for every signature
H   = SHA-256(M')
DB  = 00 … 00 ‖ 01 ‖ salt
EM  = (DB ⊕ MGF1(H)) ‖ H ‖ BC                        s = EM^d mod n
```

The **Probabilistic Signature Scheme** has a security proof (forging is as hard as inverting RSA), and the random salt means signing the same document twice gives two different signatures. HVMC uses PSS with SHA-256, MGF1-SHA-256 and a 32-byte salt for all new document signatures, as recommended by RFC 8017 and FIPS 186-5. The signer also re-checks each signature ($s^e = \mathrm{EM}$) before releasing it: a hardware fault during CRT signing would otherwise produce a faulty signature that reveals a prime factor of n (the Boneh-DeMillo-Lipton fault attack).

```
def emsa_pss_encode(m_hash, em_bits, salt_len=32, hash_name="sha256", _salt=None):
    """EMSA-PSS-ENCODE (RFC 8017 section 9.1.1).
        M'  = 0x00*8 || mHash || salt
        H   = Hash(M')
        DB  = PS (zeros) || 0x01 || salt
        EM  = (DB xor MGF1(H)) || H || 0xbc
    The random salt makes every signature different and gives PSS its security proof."""
    h_len = len(m_hash)
    em_len = (em_bits + 7) // 8
    if em_len < h_len + salt_len + 2:
        raise SignatureError("encoding error")
    salt = _salt if _salt is not None else secrets.token_bytes(salt_len)
    h = hashlib.new(hash_name, b"\x00" * 8 + m_hash + salt).digest()
    db = b"\x00" * (em_len - salt_len - h_len - 2) + b"\x01" + salt
    masked_db = bytearray(_xor(db, mgf1(h, em_len - h_len - 1, hash_name)))
    masked_db[0] &= 0xFF >> (8 * em_len - em_bits)      # clear bits above em_bits
```

```
    return bytes(masked_db) + h + b"\xbc"

def pss_sign(priv, message, salt_len=32, hash_name="sha256"):
    """RSASSA-PSS-SIGN (RFC 8017 section 8.1.1): hash, encode, then s = EM^d mod n."""
    mod_bits = priv.n.bit_length()
    em = emsa_pss_encode(hashlib.new(hash_name, message).digest(), mod_bits - 1, salt_len, hash_name)
    s = priv.decrypt_raw(os2ip(em))              # the private-key operation (CRT + blinding)
    if pow(s, priv.e, priv.n) != os2ip(em):       # fault check: a CRT fault would leak a prime factor
        raise SignatureError("signature self-check failed")
    return i2osp(s, priv.k)

def emsa_pss_verify(m_hash, em, em_bits, salt_len=32, hash_name="sha256"):
    """EMSA-PSS-VERIFY (RFC 8017 section 9.1.2). Returns True/False."""
    h_len = len(m_hash)
    em_len = (em_bits + 7) // 8
    if len(em) != em_len or em_len < h_len + salt_len + 2 or em[-1] != 0xBC:
        return False
    masked_db, h = em[:em_len - h_len - 1], em[em_len - h_len - 1:-1]
    top = 0xFF << (8 - (8 * em_len - em_bits)) & 0xFF
    if masked_db[0] & top:
        return False
    db = bytearray(_xor(masked_db, mgf1(h, em_len - h_len - 1, hash_name)))
    db[0] &= 0xFF >> (8 * em_len - em_bits)
    ps_len = em_len - h_len - salt_len - 2
    if db[:ps_len] != b"\x00" * ps_len or db[ps_len] != 0x01:
        return False
    salt = bytes(db[-salt_len:])
    return secrets.compare_digest(hashlib.new(hash_name, b"\x00" * 8 + m_hash + salt).digest(), h)
```

3.4 Results and interoperability

```
=== Demo 1: RSA signatures from scratch ===
  RSA-PSS-SHA256 signature by Sarah Chen's RSA-3072 key: 384 bytes, 17 ms; verifies = True
  signing the same document again gives a different signature (random salt) = True
  PKCS #1 v1.5 signature is deterministic and identical to the library's = True
  textbook RSA (no hash, no padding): signatures on m1 and m2 multiply into a valid signature on m1*m2 = True -> existential forgery; hashing + PSS
padding stop this
```

Check	Result
From-scratch PSS signature verified by the **cryptography** library	**PASS**
Library PSS signatures (32-byte and maximum-length salt) verified by the from-scratch code	**PASS**
From-scratch PSS signature verified by **OpenSSL** `dgst -sigopt rsa_padding_mode:pss`	**PASS**
From-scratch PKCS #1 v1.5 signature identical to the library's (byte for byte)	**PASS**
From-scratch PKCS #1 v1.5 signature identical to OpenSSL's (byte for byte)	**PASS**
Modified message, modified signature, wrong public key, textbook forgery — all rejected by PSS	**PASS**

A PSS signature with RSA-3072 is 384 bytes and took about 17 ms in pure Python.

4. Certificate Generation and Management

4.1 Certificate profiles

Each kind of certificate is issued from a **profile** that fixes its key usage, extended key usage, lifetime and policy. A certificate for one purpose cannot be used for another, because verifiers check these fields.

Profile	Key	Key usage	Extended key usage	Lifetime	Policy
Document signing	RSA-3072	digitalSignature, **nonRepudiation** (critical)	**documentSigning** (RFC 9336), emailProtection	2 years (capped at a contractor's end date)	HVMC Clinical Document Signing Policy
Time-stamping	RSA-3072	digitalSignature, nonRepudiation	**timeStamping only, critical** (RFC 3161 §2.3)	5 years	same
TLS server (rubric 4)	RSA-2048	digitalSignature, keyEncipherment	serverAuth	397 days	—
Client authentication (rubric 4)	RSA-2048	digitalSignature, keyEncipherment	clientAuth, emailProtection	365 days	—

Key separation. Sarah Chen has two key pairs: the rubric 4 key for *authenticating* to systems, and a new key for *signing* documents. If an authentication key is used by a system on her behalf, that system could also produce signatures she never approved, undermining non-repudiation. A signing key is used only for signing and, in production, stays on her smart card. The test `test_key_separation` confirms the fingerprints differ.

Certificate policy. Every signing certificate carries the HVMC policy OID `2.25.301168339931612099844041 6…`, created under the ITU-T X.667 UUID arc (2.25), which needs no registration. The policy points to the Certification Practice Statement and tells relying parties under which rules the certificate was issued; the verifier refuses certificates without it.

4.2 Issuance with approval

Each signer creates a key pair and a CSR; the CA checks the CSR signature, key size and that the email is under `@hvmc.example`, and records **who requested and who approved** each certificate in `pki/ca_register.json`. The registration authority follows the IAM project: Information Security (Denise Coleman) approves clinicians' signing certificates, and a contractor's certificate is approved by his **sponsor** (Kevin O'Brien). Carlos Diaz's certificate is **capped at his contract end date**, 2026-12-31 — the same leave date recorded in Entra ID — instead of the standard 2 years.

```
    def issue_signing(self, csr, profile, now, requested_by, approved_by, not_after_cap=None):
        """Document-signing or time-stamping certificate. The registration authority is the
        signer's department lead (the Entra ID manager set in the IAM project)."""
        sans = self.check_policy(csr, profile if profile != "document-signing" else "client-auth")
        not_after = now + dt.timedelta(days=PROFILE_DAYS[profile])
        if not_after_cap and not_after_cap < not_after:
            not_after = not_after_cap                    # e.g. a contractor's end date
        if profile == "document-signing":
            ku = pki._ku(digital_signature=True, content_commitment=True)    # contentCommitment = nonRepudiation
            eku = x509.ExtendedKeyUsage([DOCUMENT_SIGNING, ExtendedKeyUsageOID.EMAIL_PROTECTION])
            eku_critical = False
        else:
            ku = pki._ku(digital_signature=True, content_commitment=True)
            eku = x509.ExtendedKeyUsage([ExtendedKeyUsageOID.TIME_STAMPING])
            eku_critical = True                          # RFC 3161 section 2.3: must be critical and the only EKU
        policies = x509.CertificatePolicies([x509.PolicyInformation(HVMC_DOC_POLICY, [CPS_URL])])
        cert = (x509.CertificateBuilder()
                .subject_name(csr.subject).issuer_name(self.cert.subject)
                .public_key(csr.public_key()).serial_number(pki.serial())
                .not_valid_before(now - dt.timedelta(minutes=5)).not_valid_after(not_after)
                .add_extension(x509.BasicConstraints(ca=False, path_length=None), critical=True)
                .add_extension(ku, critical=True)
                .add_extension(eku, critical=eku_critical)
                .add_extension(sans, critical=False)
                .add_extension(policies, critical=False)
                .add_extension(x509.SubjectKeyIdentifier.from_public_key(csr.public_key()), critical=False)
                .add_extension(x509.AuthorityKeyIdentifier.from_issuer_public_key(self.key.public_key()), critical=False)
                .add_extension(x509.CRLDistributionPoints([x509.DistributionPoint(
                    [x509.UniformResourceIdentifier(pki.CRL_URL)], None, None, None)]), critical=False)
                .sign(self.key, hashes.SHA256()))
        self._record(cert, profile, requested_by, approved_by)
        return cert
```

Subject	Department	Key	Extended key usage	Valid until	Approved by
Sarah Chen	Doctors	RSA-3072	documentSigning, emailProtection	2028-10-25	Denise Coleman (Information Security)
Angela Brooks	Nursing	RSA-3072	documentSigning, emailProtection	2028-10-25	Denise Coleman (Information Security)
David Patel	Pharmacy	RSA-3072	documentSigning, emailProtection	2028-10-25	Denise Coleman (Information Security)
Carlos Diaz	IT	RSA-3072	documentSigning, emailProtection	2026-12-31	Kevin O'Brien (sponsor)
HVMC Time-Stamping Authority 1	Information Security	RSA-3072	timeStamping (critical)	2031-10-25	Denise Coleman (Information Security)

4.3 Lifecycle: revocation, re-key and renewal

The demo moves through a simulated timeline that starts when the script runs (T0 = 2026-10-26 17:57 UTC). Three certificates are revoked, each with an **RFC 5280 reason code**:

Event	Action	Reason code
T0+4d — **JML-203**: contractor Carlos Diaz leaves (Entra account disabled, sessions revoked)	Signing certificate revoked; key inventory updated	`cessationOfOperation`
T0+7d — Angela Brooks reports her laptop stolen on T0+6d	Revoked with an **invalidity date** of T0+6d (when the key became unsafe); new key pair and certificate issued (re-key)	`keyCompromise`
T0+9d — portal certificate flagged by the expiry monitor	Re-keyed: new RSA-2048 key and certificate; old certificate revoked after cut-over; old key retired	`superseded`

Expiry monitoring. `expiry_report()` lists valid certificates that expire inside a renewal window. Run for 2027-11-27 minus 30 days, it flags `portal.hvmc.example` with 30 days left; HVMC renews 30 days ahead, always with a **new key** (re-key), never by re-certifying the old one.

```
    def revoke(self, cert, reason, at, note, invalidity=None, replaced_by=None):
        """reason is an x509.ReasonFlags value. For keyCompromise the invalidity date records
        when the key is believed to have been compromised (may be before the revocation)."""
        rec = self.find(cert)
        if rec["status"] == "revoked":
            raise ValueError("already revoked")
        rec.update({"status": "revoked", "revoked_at": at.isoformat(), "reason": reason.value,
                    "invalidity_date": invalidity.isoformat() if invalidity else None, "note": note,
                    "replaced_by": replaced_by})
        self._save()
        return rec

    def crl(self, now, days=7):
        """CRL with a reason code and, for key compromise, an invalidity date for each entry."""
        self.state["crl_number"] += 1
        with open(self.state_path, "w") as f:
            json.dump(self.state, f)
        b = (x509.CertificateRevocationListBuilder().issuer_name(self.cert.subject)
             .last_update(now).next_update(now + dt.timedelta(days=days))
             .add_extension(x509.CRLNumber(self.state["crl_number"]), critical=False)
             .add_extension(x509.AuthorityKeyIdentifier.from_issuer_public_key(self.key.public_key()), critical=False))
        for r in self.register:
            if r["status"] == "revoked" and dt.datetime.fromisoformat(r["revoked_at"]) <= now:
                entry = (x509.RevokedCertificateBuilder().serial_number(int(r["serial"], 16))
                         .revocation_date(dt.datetime.fromisoformat(r["revoked_at"]))
                         .add_extension(x509.CRLReason(x509.ReasonFlags(r["reason"])), critical=False))
                if r["invalidity_date"]:
                    entry = entry.add_extension(x509.InvalidityDate(
                        dt.datetime.fromisoformat(r["invalidity_date"]).replace(tzinfo=None)), critical=False)
                b = b.add_revoked_certificate(entry.build())
        return b.sign(self.key, hashes.SHA256())
```

4.4 The published CRL

Certificate	Revoked (UTC)	Reason	Invalidity date
portal.hvmc.example	2026-11-04 19:57	`superseded`	—
Angela Brooks	2026-11-02 17:57	`keyCompromise`	2026-11-01 17:57
Carlos Diaz	2026-10-30 17:57	`cessationOfOperation`	—

```
$ openssl crl -in crl/hvmc-issuing-ca-1.crl.pem -noout -text
Certificate Revocation List (CRL):
        Version 2 (0x1)
    Signature Algorithm: sha256WithRSAEncryption
        Issuer: /C=US/O=Horizon Valley Medical Center/OU=Information Security/CN=HVMC Issuing CA 1
        Last Update: Nov  5 17:57:25 2026 GMT
        Next Update: Nov 12 17:57:25 2026 GMT
        CRL extensions:
            X509v3 CRL Number:
                8
            X509v3 Authority Key Identifier:
                keyid:79:35:F1:D5:4C:AF:AC:6B:B1:95:63:2E:D7:83:6F:12:98:E4:E9:70

Revoked Certificates:
    Serial Number: 5729F24E8FEA641B46B15C08CE88A0335DE93715
        Revocation Date: Nov  4 19:57:25 2026 GMT
        CRL entry extensions:
            X509v3 CRL Reason Code:
                Superseded
    Serial Number: 6A8880EEAE1DA54427417C514B7ED2B3ABA653D3
        Revocation Date: Nov  2 17:57:25 2026 GMT
        CRL entry extensions:
            X509v3 CRL Reason Code:
                Key Compromise
            Invalidity Date:
                Nov  1 17:57:25 2026 GMT
    Serial Number: 505D36E31782C2027E02A05AEE51B2139916D0EB
        Revocation Date: Oct 30 17:57:25 2026 GMT
        CRL entry extensions:
            X509v3 CRL Reason Code:
                Cessation Of Operation
    Signature Algorithm: sha256WithRSAEncryption
```

OpenSSL checks each certificate against this CRL (`-crl_check`, with `-attime` set to the simulated publication time): Sarah's signing certificate and the renewed portal certificate are **OK**; Carlos's and the old portal certificate fail with **error 23, certificate revoked**:

```
$ openssl verify -attime 1793905045 -CAfile certs/hvmc-root-ca-r1.crt.pem -untrusted certs/hvmc-issuing-ca-1.crt.pem -crl_check -CRLfile crl/hvmc-
issuing-ca-1.crl.pem certs/sarah.chen.sign.crt.pem
certs/sarah.chen.sign.crt.pem: OK
$ openssl verify -attime 1793905045 -CAfile certs/hvmc-root-ca-r1.crt.pem -untrusted certs/hvmc-issuing-ca-1.crt.pem -crl_check -CRLfile crl/hvmc-
issuing-ca-1.crl.pem certs/carlos.diaz.sign.crt.pem
C = US, O = Horizon Valley Medical Center, OU = IT, CN = Carlos Diaz, emailAddress = carlos.diaz@hvmc.example
error 23 at 0 depth lookup:certificate revoked
certs/carlos.diaz.sign.crt.pem: verification failed: 23 (certificate revoked)
$ openssl verify -attime 1793905045 -CAfile certs/hvmc-root-ca-r1.crt.pem -untrusted certs/hvmc-issuing-ca-1.crt.pem -crl_check -CRLfile crl/hvmc-
issuing-ca-1.crl.pem certs/portal.hvmc.example.crt.pem
C = US, O = Horizon Valley Medical Center, OU = IT Infrastructure, CN = portal.hvmc.example
error 23 at 0 depth lookup:certificate revoked
certs/portal.hvmc.example.crt.pem: verification failed: 23 (certificate revoked)
$ openssl verify -attime 1793905045 -CAfile certs/hvmc-root-ca-r1.crt.pem -untrusted certs/hvmc-issuing-ca-1.crt.pem -crl_check -CRLfile crl/hvmc-
issuing-ca-1.crl.pem -purpose sslserver certs/portal.hvmc.example.renewed.crt.pem
certs/portal.hvmc.example.renewed.crt.pem: OK
```

4.5 A document-signing certificate (OpenSSL view)

```
$ openssl x509 -in certs/sarah.chen.sign.crt.pem -noout -text
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            20:d3:72:46:c9:77:04:d6:a1:b1:dd:0f:be:99:f0:b1:e8:ce:7b:5b
    Signature Algorithm: sha256WithRSAEncryption
        Issuer: C=US, O=Horizon Valley Medical Center, OU=Information Security, CN=HVMC Issuing CA 1
        Validity
            Not Before: Oct 26 17:52:25 2026 GMT
            Not After : Oct 25 17:57:25 2028 GMT
        Subject: C=US, O=Horizon Valley Medical Center, OU=Doctors, CN=Sarah Chen/emailAddress=sarah.chen@hvmc.example
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                RSA Public-Key: (3072 bit)
                Exponent: 65537 (0x10001)
        X509v3 extensions:
            X509v3 Basic Constraints: critical
                CA:FALSE
            X509v3 Key Usage: critical
                Digital Signature, Non Repudiation
            X509v3 Extended Key Usage:
                1.3.6.1.5.5.7.3.36, E-mail Protection
            X509v3 Subject Alternative Name:
                email:sarah.chen@hvmc.example
            X509v3 Certificate Policies:
                Policy: 2.25.301168339931612099844041656523656322233
                  CPS: http://pki.hvmc.example/cps/hvmc-certificate-practice-statement-v1.pdf

            X509v3 Subject Key Identifier:
                FC:08:B9:83:60:D5:4C:56:DA:75:CB:B0:D0:75:96:AA:86:25:9C:14
            X509v3 Authority Key Identifier:
                keyid:79:35:F1:D5:4C:AF:AC:6B:B1:95:63:2E:D7:83:6F:12:98:E4:E9:70

            X509v3 CRL Distribution Points:

                Full Name:
                  URI:http://pki.hvmc.example/crl/hvmc-issuing-ca-1.crl

    Signature Algorithm: sha256WithRSAEncryption
```

5. The Signature Verification Process

Checking the RSA equation is only one step. A relying party must also decide whether it trusts the signer's certificate, whether that certificate was valid and unrevoked *when the signature was made*, and whether the signer was allowed to sign that kind of document. `Verifier.verify()` runs these checks in order and stops at the first failure:

#	Check	What a failure means
1	Algorithm is on the trust store's accepted list (RSASSA-PSS-SHA256)	Downgrade to a weak or unexpected algorithm
2	SHA-256 of the document equals `doc_sha256` in the signed attributes	Document changed after signing
3	SHA-256 of the attached signer certificate equals `signer_cert_sha256`	Certificate swapped for another person's
4	RSA-PSS signature over the canonical signed attributes (from-scratch *and* library)	Attributes edited, forged signature, wrong key
5	Timestamp token: imprint = SHA-256(signature), TSA signature, timeStamping EKU	Fake or re-used timestamp
6	TSA certificate path to the pinned root and not revoked at the stamp time	Untrusted time source
7	Signer certificate path to the **pinned** root, valid **at the trusted signing time**	Impostor CA; certificate expired or not yet valid when signed
8	Key usage digitalSignature + nonRepudiation; EKU documentSigning	TLS or e-mail certificate used to sign records
9	Certificate policy = HVMC Clinical Document Signing Policy	Certificate issued under different rules
10	CRL signed by the issuing CA and current	Forged or stale revocation data

#	Check	What a failure means
11	Revocation judged at signing time: signed before the revocation date (or before the invalidity date for keyCompromise)	Signature made after the signer left or the key was stolen
12	Signer's department is authorised for this document type	Valid signer acting outside their role

The result is one of three values, following the ETSI signature-validation model (EN 319 102-1): **VALID** (every check passed), **INVALID** (a check failed — the signature must not be relied on), or **INDETERMINATE** (nothing is wrong, but the evidence is not enough to decide).

5.1 Time matters: revocation and long-term validation

Revoking a certificate must not invalidate everything its owner ever signed. Carlos's change record from his last week is still a genuine record after his contract ends. So the verifier asks: **was the signature made before the cut-off?** The cut-off is the revocation date, or for `keyCompromise` the earlier **invalidity date**, because anything signed after the key was stolen may be the thief's work. This only works if the signing time can be trusted — a signer's own clock could be set back — which is why every signature gets a **TSA timestamp**. The same timestamp lets a signature be validated years later, after the signer's certificate has expired (long-term validation): without it, the verifier can only answer INDETERMINATE.

```
    def _revocation(self, cert, at_time, now, r, label):
        """Revocation only invalidates signatures made after the cut-off: the revocation date,
        or for key compromise the (earlier) invalidity date."""
        crl = self.crl_provider(now)
        issuer = next(c for c in self.intermediates if c.subject == cert.issuer)
        if not r.add(f"{label}: CRL #{crl.extensions.get_extension_for_class(x509.CRLNumber).value.crl_number} "
                     f"signed by the issuing CA and current", crl.is_signature_valid(issuer.public_key())
                     and crl.next_update_utc >= now, f"next update {crl.next_update_utc:%Y-%m-%d}"):
            return False
        state, info = cm.status(crl, cert)
        if state == "good":
            return r.add(f"{label}: revocation status", True, "not revoked")
        cutoff = info["invalidity_date"] or info["revoked_at"]
        ok = at_time < cutoff
        return r.add(f"{label}: revocation status", ok,
                     f"revoked {info['revoked_at']:%Y-%m-%d %H:%M} ({info['reason'].value})"
                     + (f", key invalid from {info['invalidity_date']:%Y-%m-%d %H:%M}" if info["invalidity_date"] else "")
                     + (" - signed BEFORE the cut-off, so the signature stays valid" if ok else " - signed AFTER the cut-off"))
```

5.2 Thirteen scenarios

#	Scenario	Result	Reason
1	Discharge summary signed by Sarah Chen, verified at T0+2h	**VALID**	all checks passed
2	Dose changed from 875 mg to 1750 mg after signing	**INVALID**	document was modified after signing
3	Signed attribute edited (document moved to another patient's record)	**INVALID**	signature does not verify
4	Signer certificate swapped for another doctor's	**INVALID**	signer certificate was substituted
5	A nurse signs a discharge summary (valid key, wrong authority)	**INVALID**	signer not authorised for this document type
6	Attacker's own 'HVMC Root CA R1' issues a 'Sarah Chen' certificate	**INVALID**	signer certificate not trusted at signing time
7	Carlos's change record signed at T0+1h, verified after his revocation	**VALID**	all checks passed
8	Carlos still has his key file and signs a new change record at T0+5d	**INVALID**	signer certificate was revoked before the signature was made
9	Note signed with Angela's stolen key at T0+6d12h (before anyone knew)	**INVALID**	signer certificate was revoked before the signature was made
10	Angela's genuine note from T0+1h, verified after the compromise	**VALID**	all checks passed
11	Angela signs with her NEW key and certificate at T0+8d	**VALID**	all checks passed
12	Same document, signature WITH a TSA timestamp	**VALID**	all checks passed
13	Same document, signature WITHOUT a timestamp	**INDETERMINATE**	signer certificate has expired and there is no trusted timestamp proving the signature was made while it was valid

5.3 Verification traces

Scenario 1, the normal case — every check passes:

```
  [VALID] 1. Discharge summary signed by Sarah Chen, verified at T0+2h
      signer: Sarah Chen (Doctors); trusted signing time: 2026-10-26 18:57 UTC; all checks passed
      pass  algorithm accepted by the trust policy  (RSASSA-PSS-SHA256)
      pass  document SHA-256 matches the signed attribute  (fd779d4597eb9cb994d4c70d…)
      pass  signer certificate matches the signed certificate hash  (Sarah Chen (Doctors))
      pass  RSA-PSS signature over the signed attributes  (from-scratch=True, library=True)
      pass  timestamp token: imprint, TSA signature, timeStamping EKU  (HVMC Time-Stamping Authority 1, gen_time 2026-10-26 18:57 UTC)
      pass  TSA: certificate path to a pinned HVMC trust anchor (at 2026-10-26 18:57 UTC)  (11 checks)
      pass  TSA: CRL #2 signed by the issuing CA and current  (next update 2026-11-02)
      pass  TSA: revocation status  (not revoked)
      pass  signer: certificate path to a pinned HVMC trust anchor (at 2026-10-26 18:57 UTC)  (11 checks)
      pass  key usage digitalSignature + nonRepudiation, EKU documentSigning
      pass  certificate policy = HVMC Clinical Document Signing Policy  (2.25.30116833993161209…)
      pass  signer: CRL #2 signed by the issuing CA and current  (next update 2026-11-02)
      pass  signer: revocation status  (not revoked)
      pass  signer's department is authorised for 'discharge-summary'  (Doctors; allowed: Doctors)
```

Scenarios 9 and 10 — the stolen key. The forged note carries a perfect signature, but its trusted time falls after the invalidity date; Angela's genuine note from before the theft stays valid:

```
  [INVALID] 9. Note signed with Angela's stolen key at T0+6d12h (before anyone knew)
      signer: Angela Brooks (Nursing); trusted signing time: 2026-11-02 05:57 UTC; signer certificate was revoked before the signature was made
      pass  algorithm accepted by the trust policy  (RSASSA-PSS-SHA256)
      pass  document SHA-256 matches the signed attribute  (8e4d1c6b42ea6e1807d36635…)
      pass  signer certificate matches the signed certificate hash  (Angela Brooks (Nursing))
      pass  RSA-PSS signature over the signed attributes  (from-scratch=True, library=True)
      pass  timestamp token: imprint, TSA signature, timeStamping EKU  (HVMC Time-Stamping Authority 1, gen_time 2026-11-02 05:57 UTC)
      pass  TSA: certificate path to a pinned HVMC trust anchor (at 2026-11-02 05:57 UTC)  (11 checks)
      pass  TSA: CRL #5 signed by the issuing CA and current  (next update 2026-11-09)
      pass  TSA: revocation status  (not revoked)
      pass  signer: certificate path to a pinned HVMC trust anchor (at 2026-11-02 05:57 UTC)  (11 checks)
      pass  key usage digitalSignature + nonRepudiation, EKU documentSigning
      pass  certificate policy = HVMC Clinical Document Signing Policy  (2.25.30116833993161209…)
      pass  signer: CRL #5 signed by the issuing CA and current  (next update 2026-11-09)
      FAIL  signer: revocation status  (revoked 2026-11-02 17:57 (keyCompromise), key invalid from 2026-11-01 17:57 - signed AFTER the cut-off)

  [VALID] 10. Angela's genuine note from T0+1h, verified after the compromise
      signer: Angela Brooks (Nursing); trusted signing time: 2026-10-26 18:57 UTC; all checks passed
      pass  algorithm accepted by the trust policy  (RSASSA-PSS-SHA256)
      pass  document SHA-256 matches the signed attribute  (7e3c29b0804f93246781ef81…)
      pass  signer certificate matches the signed certificate hash  (Angela Brooks (Nursing))
      pass  RSA-PSS signature over the signed attributes  (from-scratch=True, library=True)
      pass  timestamp token: imprint, TSA signature, timeStamping EKU  (HVMC Time-Stamping Authority 1, gen_time 2026-10-26 18:57 UTC)
      pass  TSA: certificate path to a pinned HVMC trust anchor (at 2026-10-26 18:57 UTC)  (11 checks)
      pass  TSA: CRL #5 signed by the issuing CA and current  (next update 2026-11-09)
      pass  TSA: revocation status  (not revoked)
      pass  signer: certificate path to a pinned HVMC trust anchor (at 2026-10-26 18:57 UTC)  (11 checks)
      pass  key usage digitalSignature + nonRepudiation, EKU documentSigning
      pass  certificate policy = HVMC Clinical Document Signing Policy  (2.25.30116833993161209…)
      pass  signer: CRL #5 signed by the issuing CA and current  (next update 2026-11-09)
      pass  signer: revocation status  (revoked 2026-11-02 17:57 (keyCompromise), key invalid from 2026-11-01 17:57 - signed BEFORE the cut-off, so the
signature stays valid)
      pass  signer's department is authorised for 'nursing-note'  (Nursing; allowed: Nursing)
```

Scenarios 12 and 13 — the same discharge summary checked 30 days after Sarah's certificate expired, with and without a timestamp:

```
  [VALID] 12. Same document, signature WITH a TSA timestamp
      signer: Sarah Chen (Doctors); trusted signing time: 2026-10-26 18:57 UTC; all checks passed
      pass  algorithm accepted by the trust policy  (RSASSA-PSS-SHA256)
      pass  document SHA-256 matches the signed attribute  (fd779d4597eb9cb994d4c70d…)
      pass  signer certificate matches the signed certificate hash  (Sarah Chen (Doctors))
      pass  RSA-PSS signature over the signed attributes  (from-scratch=True, library=True)
      pass  timestamp token: imprint, TSA signature, timeStamping EKU  (HVMC Time-Stamping Authority 1, gen_time 2026-10-26 18:57 UTC)
      pass  TSA: certificate path to a pinned HVMC trust anchor (at 2026-10-26 18:57 UTC)  (11 checks)
      pass  TSA: CRL #7 signed by the issuing CA and current  (next update 2028-12-01)
      pass  TSA: revocation status  (not revoked)
      pass  signer: certificate path to a pinned HVMC trust anchor (at 2026-10-26 18:57 UTC)  (11 checks)
      pass  key usage digitalSignature + nonRepudiation, EKU documentSigning
      pass  certificate policy = HVMC Clinical Document Signing Policy  (2.25.30116833993161209…)
      pass  signer: CRL #7 signed by the issuing CA and current  (next update 2028-12-01)
      pass  signer: revocation status  (not revoked)
      pass  signer's department is authorised for 'discharge-summary'  (Doctors; allowed: Doctors)

  [INDETERMINATE] 13. Same document, signature WITHOUT a timestamp
      signer: Sarah Chen (Doctors); trusted signing time: none; signer certificate has expired and there is no trusted timestamp proving the signature
was made while it was valid
      pass  algorithm accepted by the trust policy  (RSASSA-PSS-SHA256)
      pass  document SHA-256 matches the signed attribute  (fd779d4597eb9cb994d4c70d…)
      pass  signer certificate matches the signed certificate hash  (Sarah Chen (Doctors))
      pass  RSA-PSS signature over the signed attributes  (from-scratch=True, library=True)
      pass  timestamp token  (none - only the signer's own claimed time, which cannot be trusted)
      FAIL  signer: certificate path to a pinned HVMC trust anchor (at 2028-11-24 17:57 UTC)  (11 checks; failed: Sarah Chen: inside validity period)
```

6. Trust Model Implementation

A **trust model** answers: *why should a relying party believe that this public key belongs to this person?* HVMC uses a **hierarchical (single-root) PKI**: every relying party trusts one root, and trust flows down signed certificates. This fits a hospital well: there is one organisation, one HR/IAM system that knows who works where, and one security team accountable for issuing identities.

6.1 Roles

Role	HVMC holder	Responsibility
Policy authority	Information Security (Denise Coleman), approved by the CISO	Owns the certificate policy (OID above) and the Certification Practice Statement; approves profiles and the document authorisation matrix
Root CA	HVMC Root CA R1 — offline	Signs only the issuing CA certificate and its own CRL; key ceremony with two people present
Issuing CA	HVMC Issuing CA 1	Issues signing, TSA, TLS and client certificates; publishes a CRL weekly and after every revocation
Registration authority	Information Security; contractor sponsors	Checks identity and approves each request; identity comes from the Entra ID record (employee ID, department, manager, leave date)
Subscribers	Clinicians and staff	Protect their signing key (smart card in production), report loss at once
Time-stamping authority	HVMC TSA 1	Provides trusted time for every signature
Relying parties	EHR, pharmacy system, auditors	Verify every signature with the process in section 5 and the trust store below

6.2 What the implementation enforces

Trust decision	How it is implemented	Code
Which root to trust	One anchor, **pinned by SHA-256 fingerprint**. A look-alike root with the same name (scenario 6) is rejected because signatures do not chain to the pinned key	`TrustStore`
Distribution of the anchor	Pushed to managed devices (Intune / Group Policy); fingerprint confirmed out of band before first use	`trust_store.json`
Which certificates count	Must carry the HVMC certificate policy, documentSigning EKU and nonRepudiation	`Verifier.verify` steps 8–9
Which algorithms	RSASSA-PSS-SHA256 only for new signatures	step 1
Who may sign what	Authorisation matrix by department (certificate OU): only Doctors sign discharge summaries; Pharmacy or Doctors sign medication verifications	`DOCUMENT_POLICY`
When trust ends	CRL revocation with reason codes; invalidity date for compromised keys; contractor certificates end with the contract	`ManagedCA.revoke` , `crl`
Which time to believe	Only the TSA's time; the signer's claimed time is recorded but never trusted	steps 5–7, 11


```
{
  "anchors": [
    {
      "subject": "CN=HVMC Root CA R1,OU=Information Security,O=Horizon Valley Medical Center,C=US",
      "sha256": "4e0f2629820c5028add49dd796fa47d8e68765deeb4c2acf1d7d1d92374ecbea",
      "not_after": "2036-10-23"
    }
  ],
  "required_certificate_policy": "2.25.301168339931612099844041656523656322233",
  "accepted_algorithms": [
    "RSASSA-PSS-SHA256"
  ],
  "document_policy": {
    "discharge-summary": [
      "Doctors"
    ],
    "nursing-note": [
      "Nursing"
    ],
    "medication-verification": [
      "Doctors",
      "Pharmacy"
    ],
    "change-record": [
      "IT",
      "Information Security"
    ]
```

```
  }
}
```

6.3 Why hierarchical, compared with other trust models

Model	How trust is established	Fit for HVMC
Hierarchical PKI (chosen)	A root CA vouches for issuing CAs, which vouch for people	Central control, clear accountability, revocation in one place, links to IAM joiner/mover/leaver
Web of trust (PGP)	Users sign each other's keys; trust is personal and transitive	No central revocation or accountability; unsuitable for regulated clinical records
Direct trust / pinning (SSH known_hosts)	Each relying party stores each key it trusts	Does not scale to 30+ signers and many systems; no expiry or revocation
Bridge / cross-certified PKI	CAs of different organisations certify each other	A future option for signing with partner hospitals or labs; adds policy-mapping complexity

6.4 Threats and mitigations

Threat	Mitigation	Shown in
Document altered after signing	Hash in signed attributes	Scenario 2
Signature moved to another patient's record	Document ID signed with the hash	Scenario 3
Certificate substitution	Signer certificate hash is a signed attribute	Scenario 4
Insider signs outside their role	Authorisation matrix	Scenario 5
Rogue or look-alike CA	Pinned trust anchor	Scenario 6
Leaver keeps using a key	Revocation tied to JML; TSA time vs revocation date	Scenarios 7–8
Stolen key	keyCompromise + invalidity date; re-key	Scenarios 9–11
Back-dating a signature	Only TSA time is trusted	Section 5.1
Records outlive certificates	Timestamps for long-term validation	Scenarios 12–13
Compromise of the issuing CA	Root revokes it on the root CRL; offline root rebuilds a new issuing CA	Section 6.1

7. Best Practices and Limitations

- **Use PSS** for new signatures; keep PKCS #1 v1.5 only where formats require it (X.509, older systems).
- **Sign attributes, not just bytes:** bind the document hash, its ID and the signer certificate together.
- **Separate keys by purpose** (signing vs authentication vs encryption) and keep signing keys on hardware.
- **Always timestamp** signatures that must remain verifiable after the certificate expires.
- **Tie certificate lifecycle to IAM:** contractor end dates cap certificates, and leavers are revoked in the JML process.
- **Revoke fast and precisely:** reason codes and invalidity dates let old genuine signatures survive a compromise.
- **Re-key on renewal** and monitor expiry 30+ days ahead.

7.1 Limitations of this lab

- The envelope and timestamp token are JSON models of CMS/CAdES and RFC 3161, not their DER formats, so they are not readable by Adobe or other signature tools; production HVMC would use PAdES for PDFs or CAdES for other documents.
- From-scratch code is not constant-time; production uses a vetted library with keys in an HSM or on smart cards.
- Later events (revocations, renewal, expiry) are simulated on a timeline; OpenSSL checked them with `-attime`.
- Revocation uses CRLs only; a production deployment would add OCSP for real-time status.
- Key and certificate files live in one encrypted folder; the "offline" root and HSM are by convention only.

8. Conclusion

HVMC now has a working digital signature system on top of its PKI. RSA-PSS and PKCS #1 v1.5 signatures are implemented from first principles and agree exactly with the cryptography library and OpenSSL. Signing certificates are issued under a written policy with approval, capped by contract dates, revoked with reasons when people leave or keys are stolen, and renewed with new keys. The verification process checks far more than the RSA equation: the trust anchor, the policy, the time of signing, revocation and the signer's authority. Across 13 scenarios it accepted every

genuine signature — including ones made before a revocation or long after a certificate expired — and rejected every forgery, substitution and misuse.

Appendix A. Test Results

Run with `python3 test_and_demo.py` (Python 3.9.6 on Darwin arm64, 2026-10-26 13:57; cryptography package: 47.0.0; LibreSSL 3.3.6).

Test	Group	Result
`test_pkcs1v15_matches_library_byte_for_byte`	Signatures	**PASS**
`test_pss_encoding_structure`	Signatures	**PASS**
`test_pss_interop_with_library`	Signatures	**PASS**
`test_pss_round_trip_randomised`	Signatures	**PASS**
`test_textbook_rsa_forgery`	Signatures	**PASS**
`test_contractor_capped_and_tsa_profile`	Certificates	**PASS**
`test_document_signing_profile`	Certificates	**PASS**
`test_key_separation`	Certificates	**PASS**
`test_policy_rejects_non_hvmc_email`	Certificates	**PASS**
`test_revocation_and_crl`	Certificates	**PASS**
`test_authorisation_and_trust_anchor`	Verification	**PASS**
`test_long_term_validation`	Verification	**PASS**
`test_revocation_timing`	Verification	**PASS**
`test_valid_and_tampering`	Verification	**PASS**

Appendix B. Files

Published in `cryptography_1/digital_signatures/`. The private keys and passphrase in `pki/private/` are copied from the rubric 4 PKI by the script and excluded by `.gitignore`.

```
.gitignore
cert_management.py
openssl_checks.txt
results.json
results.txt
signatures.py
signing_system.py
test_and_demo.py
trust_store.json
pki/ca_register.json
pki/ca_state.json
pki/key_inventory.json
pki/certs/angela.brooks.sign-rekeyed.crt.pem
pki/certs/angela.brooks.sign.crt.pem
pki/certs/carlos.diaz.sign.crt.pem
pki/certs/david.patel.sign.crt.pem
pki/certs/hvmc-chain.pem
pki/certs/hvmc-issuing-ca-1.crt.pem
pki/certs/hvmc-root-ca-r1.crt.pem
pki/certs/hvmc-tsa-1.crt.pem
pki/certs/portal.hvmc.example.crt.pem
pki/certs/portal.hvmc.example.renewed.crt.pem
pki/certs/sarah.chen.crt.pem
pki/certs/sarah.chen.sign.crt.pem
pki/crl/hvmc-issuing-ca-1.crl.pem
pki/crl/hvmc-root-ca-r1.crl.pem
pki/csr/hvmc-issuing-ca-1.csr.pem
pki/csr/portal.hvmc.example.csr.pem
pki/csr/sarah.chen.csr.pem
pki/public/angela-brooks-sign-2026.pub.pem
pki/public/angela-brooks-sign-2026b.pub.pem
pki/public/carlos-diaz-sign-2026.pub.pem
pki/public/david-patel-sign-2026.pub.pem
pki/public/ehr-data-key-v1.pub.pem
pki/public/ehr-data-key-v2.pub.pem
pki/public/hvmc-issuing-ca-1.pub.pem
pki/public/hvmc-root-ca-r1.pub.pem
pki/public/hvmc-tsa-1.pub.pem
pki/public/portal-tls-2026.pub.pem
pki/public/portal-tls-2027.pub.pem
pki/public/sarah-chen-client-2026.pub.pem
pki/public/sarah-chen-sign-2026.pub.pem
sample_documents/change_record_HVMC-IAM-CHG-104.json
sample_documents/change_record_HVMC-IAM-CHG-105.json
sample_documents/discharge_summary_HVMC-004417.json
sample_documents/medication_verification_HVMC-004417.json
sample_documents/nursing_note_HVMC-004417.json
sample_documents/nursing_note_HVMC-004417_forged.json
sample_documents/nursing_note_HVMC-004417_v2.json
signed_documents/change_record_HVMC-IAM-CHG-104.json
signed_documents/change_record_HVMC-IAM-CHG-104.sig.json
signed_documents/change_record_HVMC-IAM-CHG-105.json
signed_documents/change_record_HVMC-IAM-CHG-105.sig.json
signed_documents/discharge_summary_HVMC-004417.json
signed_documents/discharge_summary_HVMC-004417.sig.json
signed_documents/medication_verification_HVMC-004417.json
signed_documents/medication_verification_HVMC-004417.sig.json
signed_documents/nursing_note_HVMC-004417.json
signed_documents/nursing_note_HVMC-004417.sig.json
signed_documents/nursing_note_HVMC-004417_forged.json
signed_documents/nursing_note_HVMC-004417_forged.sig.json
signed_documents/nursing_note_HVMC-004417_v2.json
signed_documents/nursing_note_HVMC-004417_v2.sig.json
```

Appendix C. References

- RFC 8017 — PKCS #1 v2.2: RSASSA-PSS and RSASSA-PKCS1-v1_5; FIPS 186-5 — Digital Signature Standard
- RFC 5280 — X.509 certificate and CRL profile (reason codes, invalidity date, path validation)
- RFC 3161 — Time-Stamp Protocol; RFC 9336 — X.509 extended key usage for document signing
- RFC 5652 — Cryptographic Message Syntax; ETSI EN 319 122 (CAdES) and EN 319 102-1 (signature validation)
- ITU-T X.667 — UUID-based object identifiers (2.25 arc); NIST SP 800-57 Part 1 Rev. 5 — key management
- HIPAA Security Rule 45 CFR §164.312(c), (d); FDA 21 CFR Part 11 — electronic records and signatures